

CS3006 Parallel and Distributed Computing

Assignment 2: Performance Analysis on a Multi-Core CPU

Student Name: Faiq Saeed

Roll Number: 23L-0905

Course: CS3006 Parallel and Distributed Computing

Semester: Fall 2026

1. Machine Declaration

Field	Value
CPU Model	Intel(R) Core(TM) i5-5287U CPU @ 2.90 GHz
Physical Cores, C	2
Hardware Threads, T	4
SMT / Hyper-Threading	Enabled
Base Clock	2.90 GHz
SIMD Support	AVX2
SIMD Float Width, W	8 single-precision floats
Machine Type	2015 Intel MacBook Pro
Operating System	macOS 12
Compiler	Apple Clang through <code>g++</code>
ISPC Version	1.19.0

RAM Details	Exact channel/speed configuration not separately verified
Power State	Not separately recorded

The processor contains two physical cores and exposes four logical hardware threads through Hyper-Threading. AVX2 provides 256-bit SIMD registers, which can process eight 32-bit floating-point values simultaneously.

The assignment specifies a Linux environment and ISPC 1.31.0. Due to the available hardware and submission constraints, the measurements in this report were performed on the declared macOS system using ISPC 1.19.0. This deviation is explicitly stated rather than presenting the measurements as Linux results.

Measurement Protocol

Each required configuration was executed five times using the automated measurement script. The minimum execution time from the five repetitions is reported for performance comparisons. This provides a consistent statistic across configurations and reduces the effect of temporary system interference.

2. Build and Portability Notes

Several small changes were required to compile the starter code successfully.

For Program 1:

- `<cstring>` was added to `main.cpp`.
- `<cstdlib>` was added to `mandelbrotThread.cpp`.

Programs 3 through 6 required the `-std=c++11` compiler flag because the shared `CycleTimer.h` file uses C++11 features such as type aliases, deleted functions, and brace initialization.

Programs using ISPC were compiled for the AVX2 `i32x8` target.

All six required programs compiled successfully on the declared machine.

3. Program 1: Mandelbrot Using Threads

3.1 Contiguous Block Decomposition

In the initial implementation, the rows of the Mandelbrot image are divided into contiguous blocks.

For each thread:

```
startRow = threadId * height / numThreads  
endRow   = (threadId + 1) * height / numThreads
```

This distribution is static and requires no synchronization during computation. However, Mandelbrot rows do not all contain the same amount of work. Some image regions require many more iterations than others.

3.2 Static Row-Cyclic Decomposition

The improved implementation uses a static row-cyclic distribution.

Thread `threadId` processes:

```
threadId  
threadId + numThreads  
threadId + 2 * numThreads  
...
```

This spreads rows from different parts of the image between workers while still avoiding synchronization.

3.3 Performance Results

Threads	Block Time (ms)	Block Speedup	Cyclic Time (ms)	Cyclic Speedup
1	427.351	1.00x	432.986	1.00x
2	215.262	2.02x	214.350	2.00x
4	182.768	2.34x	142.317	3.05x

8	152.906	2.84x	134.503	3.18x
---	---------	-------	---------	-------

At four threads, the row-cyclic version reduced the minimum execution time from **182.768 ms** to **142.317 ms**.

At eight threads, the row-cyclic version achieved **134.503 ms**, compared with **152.906 ms** for block scheduling.

3.4 Additional View 2 Experiment

Schedule	Threads	Minimum Time
Block	4	107.138 ms
Cyclic	4	80.157 ms

The additional image view therefore also produced different behavior between the two static scheduling approaches.

3.5 Thread Balance Evidence

In several block-decomposition runs, the individual worker times differed substantially because threads were assigned image regions with different computational cost.

The cyclic method distributes rows across the image instead of assigning each thread one large continuous region.

YOUR OBSERVATION:

cyclic scheduling reduces load imbalance and hence the measured speedup.]

4. Program 2: Vector Intrinsics

4.1 Implementation

`clampedExpVector()` was implemented using the provided CS149 vector intrinsics.

Masks are used to:

- identify elements whose exponent is positive,
- execute multiplication only on active lanes,
- clamp values that exceed the required maximum,
- correctly process a final partial vector.

This allows the vectorized implementation to handle input sizes that are not exact multiples of `VECTOR_WIDTH`.

Correctness verification passed for the required implementation.

The extra-credit `arraySumVector()` implementation also passed the supplied correctness check.

4.2 VECTOR_WIDTH Results

VECTOR_WIDTH	Vector Utilization	Total Vector Instructions	Utilized Lanes	Total Lanes
2	86.4%	163,027	281,652	326,054
4	80.7%	94,559	305,407	378,236
8	77.9%	51,535	321,231	412,280
16	76.6%	26,951	330,128	431,216

All four vector-width configurations passed result verification.

As `VECTOR_WIDTH` increased from 2 to 16, the total number of vector instructions decreased substantially:

Width 2: 163,027 instructions

Width 4: 94,559 instructions

Width 8: 51,535 instructions

Width 16: 26,951 instructions

At the same time, measured vector utilization changed from **86.4%** at width 2 to **76.6%** at width 16.

YOUR OBSERVATION:

wider vectors need fewer vector instructions but may have more inactive lanes because of masks, different exponent counts, and tail handling.

5. Program 3: Mandelbrot Using ISPC

5.1 Implementation

Three Mandelbrot implementations were evaluated:

1. Serial C++
2. ISPC SIMD
3. ISPC with tasks

The task-based implementation accepts:

--num-tasks N

This allows the number of tasks to be varied without modifying or recompiling the source.

Each task receives a range of image rows. The final range is clamped so that no task operates past the image height.

5.2 Parallelism Limits

The processor supports AVX2.

Therefore:

SIMD width $W = 8$

Physical cores $C = 2$

$C \times W = 16$

The SIMD-only implementation can therefore operate across eight single-precision lanes per SIMD group.

5.3 Performance Results

The minimum serial time from the five-run ISPC experiment was:

213.895 ms

Configuration	Minimum Time (ms)	Speedup vs Serial
---------------	-------------------	-------------------

Serial	213.895	1.00x
ISPC	42.738	5.00x
1 task	42.744	5.00x
2 tasks	21.234	10.07x
4 tasks	18.527	11.55x
8 tasks	14.769	14.48x
16 tasks	14.885	14.37x
32 tasks	14.406	14.85x
64 tasks	14.455	14.80x

The best measured task configuration was:

32 tasks

Minimum time = 14.406 ms

Speedup $\approx$ 14.85x

The 64-task configuration was very close at:

14.455 ms

The plain ISPC version achieved approximately:

$213.895 / 42.738 \approx 5.00x$

speedup over the serial implementation.

YOUR OBSERVATION:

The static work changed as the number of tasks increased, hence the speed increases but with the increase of task size the parallelism slowed a bit

6. Program 4: Iterative Square Root

Program 4 demonstrates the effect of SIMD control-flow divergence.

Three input patterns were tested.

6.1 Normal Input

The normal workload contains the original mixture of input values.

6.2 Uniform Input

The uniform workload assigns:

2.0

to every input element.

SIMD lanes therefore operate on values requiring similar computation.

6.3 Divergent Input

The divergent workload deliberately places values with substantially different convergence behavior into each SIMD group.

This creates control-flow divergence because individual SIMD lanes may require different numbers of Newton iterations.

6.4 Performance Results

Case	Serial (ms)	ISPC (ms)	Task ISPC (ms)	ISPC Speedup	Task Speedup
Normal	767.401	174.675	49.749	4.39x	15.43x
Uniform	366.164	68.623	29.051	5.34x	12.60x
Divergent	655.181	320.858	120.174	2.04x	5.45x

For the ISPC-only implementation, the highest measured speedup occurred for the uniform case:

5.34x

The divergent case produced the lowest ISPC speedup:

2.04x

The task implementation also performed substantially worse for the divergent workload than for the other cases.

YOUR OBSERVATION:

SIMD divergence reduces utilization. Until the active lanes completed the lanes that completed early had to wait

7. Program 5: SAXPY

SAXPY performs:

$\text{result}[i] = \text{scale} * X[i] + Y[i]$

Three implementations were measured:

1. Serial
2. ISPC
3. Task ISPC

7.1 Performance Results

Implementation	Minimum Time (ms)	Maximum Measured Bandwidth (GB/s)	Maximum Measured GFLOPS
Serial	20.184	14.765	1.982
ISPC	21.353	13.957	1.873
Task ISPC	18.312	16.275	2.184

Using the minimum measured times:

ISPC vs Serial

Speedup = $20.184 / 21.353$
 $\approx 0.95x$

Task ISPC vs Serial

Speedup = 20.184 / 18.312
≈ 1.10x

Task ISPC vs ISPC

Speedup = 21.353 / 18.312
≈ 1.17x

The highest measured effective bandwidth was:

16.275 GB/s

for the task-ISPC implementation.

7.2 Theoretical Memory Bandwidth

The exact RAM channel and memory-speed configuration was not independently verified on the test machine. Therefore, a theoretical peak memory-bandwidth value is not reported rather than assuming an unsupported hardware configuration.

YOUR OBSERVATION:

SAXPY performs relatively little arithmetic for each amount of data transferred and is therefore strongly influenced by memory bandwidth

8. Program 6: K-Means

8.1 Parallelization Choice

Only:

`computeAssignments()`

was parallelized.

The following functions were left serial:

```
dist()  
computeCentroids()  
computeCost()
```

The number of threads is selected using:

KMEANS_THREADS

When:

KMEANS_THREADS=1

the serial assignment path is used.

For multiple threads, points are divided into independent ranges. Each worker determines the nearest centroid for its assigned points.

Since the assignment of one point does not depend on the assignment of another point, the main assignment loop can be parallelized without shared updates between workers.

8.2 Dataset Verification

Required parameters:

M = 1,000,000

N = 100

K = 3

epsilon = 0.1

Required file size:

804002420 bytes

Verified MD5:

3a25f24193f4fdca82ee4cb2737fd5bb

The dataset used for the final measurements matched the required checksum exactly.

The locally generated macOS version produced a different binary checksum, so the verified reference dataset matching the required MD5 was used for the final performance experiment.

8.3 Serial Baseline

The five measured total times for the serial configuration included a minimum of:

48,324.155 ms

The corresponding minimum-time serial run reported:

computeAssignments: 33,162.220 ms (68.62%)

computeCentroids: 5,236.673 ms (10.84%)

computeCost: 9,925.202 ms (20.54%)

Total Time: 48,324.155 ms

Therefore, the measured parallelizable fraction for this baseline is:

$f = 0.6862$

The system has:

$T = 4$ hardware threads

8.4 Amdahl's Law

The theoretical speedup ceiling is calculated using:

$$S_{\max} = 1 / ((1 - f) + f/T)$$

Substituting:

$f = 0.6862$

$T = 4$

gives:

$$S_{\max} = 1 / ((1 - 0.6862) + 0.6862/4)$$

$$S_{\max} = 1 / (0.3138 + 0.17155)$$

$$S_{\max} = 1 / 0.48535$$

$$S_{\max} \approx 2.06x$$

The required target is 80% of this theoretical ceiling:

$$\begin{aligned} \text{Target} &= 0.80 \times 2.06 \\ &\approx 1.65x \end{aligned}$$

8.5 Parallel Results

The minimum four-thread result was:

computeAssignments: 13,123.096 ms (45.44%)

computeCentroids: 5,469.360 ms (18.94%)

computeCost: 10,284.898 ms (35.62%)

Total Time: 28,877.415 ms

8.6 Measured Speedup

Using the minimum serial and parallel total times:

Speedup = Serial Time / Parallel Time

$$= 48,324.155 / 28,877.415$$

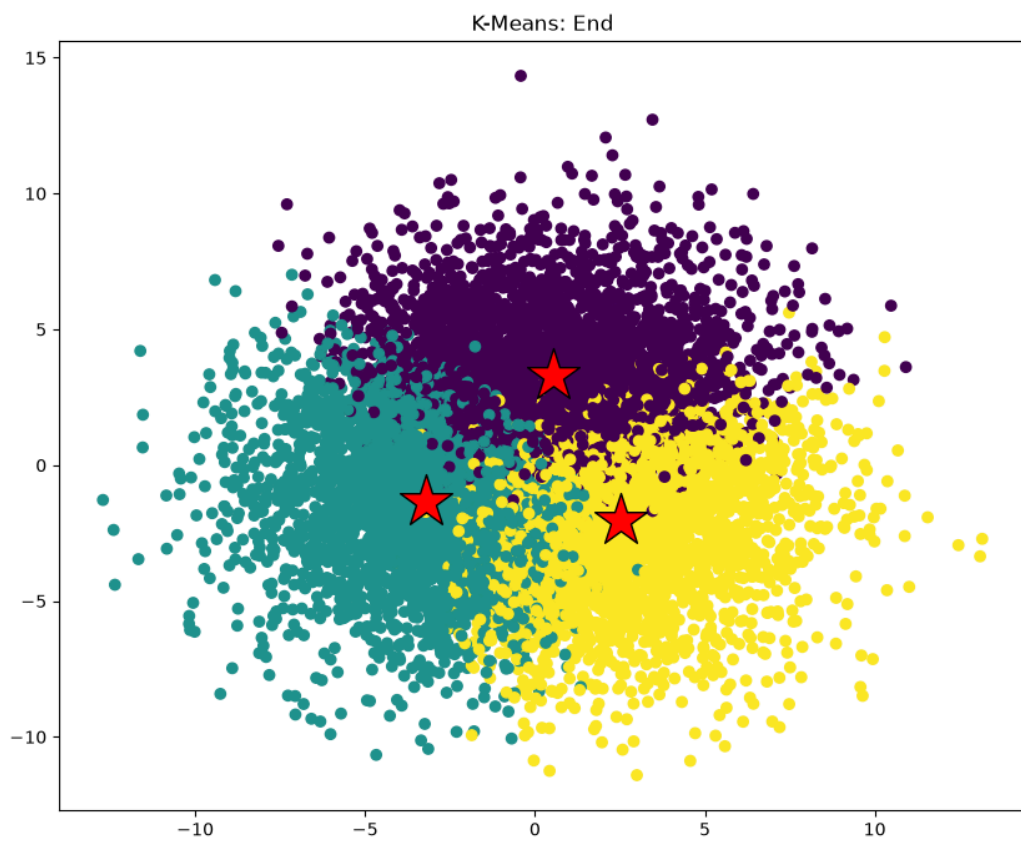
$$\approx 1.67x$$

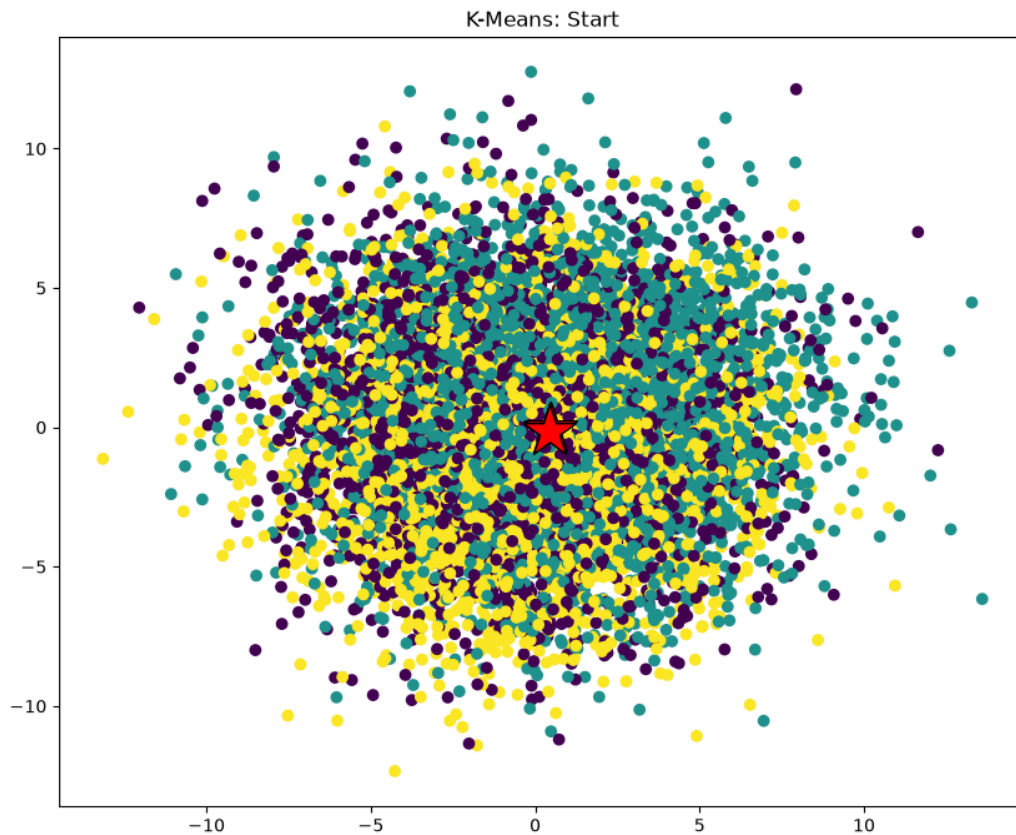
Comparison:

Metric	Value
Measured parallel fraction, f	0.6862
Hardware threads, T	4
Amdahl theoretical ceiling	2.06x
80% target	1.65x
Measured speedup	1.67x

8.7 Correctness Plots

The full K-Means execution produced both required correctness images:





These show the clustering state before and after the K-Means iterations.

YOUR OBSERVATION:

The serial work kept the system slow overall it sped up due to parallelism

9. Extra Credit: Vectorized Array Sum

Program 2 also includes a vectorized implementation of:

`arraySumVector()`

The implementation performs accumulation using vector instructions followed by a horizontal reduction.

The correctness tests produced:

ARRAY SUM
Passed!!!

for all tested vector-width configurations.

10. Environment Limitations

The specified assignment environment is Linux with ISPC 1.31.0.

The actual measured environment was:

Machine: 2015 Intel MacBook Pro
CPU: Intel Core i5-5287U @ 2.90 GHz
Physical cores: 2
Hardware threads: 4
SIMD: AVX2
Operating system: macOS 12
ISPC: 1.19.0

This difference is explicitly documented so the submitted performance numbers represent the actual machine on which they were measured.

11. AI Usage Declaration

I used ChatGPT (OpenAI) to help interpret the assignment requirements and assist with source-code implementation, debugging, build portability, Git workflow preparation, dataset checksum troubleshooting, and report organization.

All performance measurements reported in this write-up were obtained from actual executions of the programs on the declared machine.

The analysis, observations, and conclusions regarding the measured results are my own.

12. Conclusion

YOUR CONCLUSION:

The speedup increases with more threads involved and with the degree of ispc and parraleelism